

# MongoDB Basic Practice Task – Week 12

Name: Tharun Raj

## 1. Database & Collection Setup

```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.12.0
Microsoft Windows [Version 10.0.26200.9550]
(c) Microsoft Corporation. All rights reserved.

C:\Users\acer>mongosh
Current Mongosh Log ID: 6ac1286db76bbb7d310d6152
Connecting to:  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.12.0
Using MongoDB:  9.0.2
Using Mongosh:  2.12.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2026-10-01T16:02:12.265+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> use studentDB
switched to db studentDB
studentDB> db.createCollection("students")
{ ok: 1 }
studentDB> |
```

## 2. Insert One Document

```
studentDB> db.students.insertOne({
|   name: "Xavier",
|   age: 20,
|   course: "Btech",
|   status: "ongoing"
| })
{
  acknowledged: true,
  insertedId: ObjectId('6ac12bbeb76bbb7d310d6153')
}
studentDB> |
```

### 3. Insert Multiple Documents

```
studentDB> db.students.insertMany([
  {
    name: "Arun",
    age: 21,
    course: "MERN Stack",
    status: "ongoing"
  },
  {
    name: "Micheal",
    age: 17,
    course: "Full Stack",
    status: "completed"
  },
  {
    name: "Shisue",
    age: 20,
    course: "MERN Stack",
    status: "ongoing"
  },
  {
    name: "Jasmine",
    age: 21,
    course: "Cyber security",
    status: "Discontinue"
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac13020b76bbb7d310d6154'),
    '1': ObjectId('6ac13020b76bbb7d310d6155'),
    '2': ObjectId('6ac13020b76bbb7d310d6156'),
    '3': ObjectId('6ac13020b76bbb7d310d6157')
```

#### 4. Read All Documents

```
studentDB> db.students.find()

{
  _id: ObjectId('6ac12bbeb76bbb7d310d6153'),
  name: 'Xavier',
  age: 20,
  course: 'Btech',
  status: 'ongoing'
},
{
  _id: ObjectId('6ac13020b76bbb7d310d6154'),
  name: 'Arun',
  age: 21,
  course: 'MERN Stack',
  status: 'ongoing'
},
{
  _id: ObjectId('6ac13020b76bbb7d310d6155'),
  name: 'Micheal',
  age: 17,
  course: 'Full Stack',
  status: 'completed'
},
{
  _id: ObjectId('6ac13020b76bbb7d310d6156'),
  name: 'Shisue',
  age: 20,
  course: 'MERN Stack',
  status: 'ongoing'
},
```

## 5. Filter Documents

```
{
  _id: ObjectId('6ac13020b76bbb7d310d6157'),
  name: 'Jasmine',
  age: 21,
  course: 'Cyber security',
  status: 'Discontinue'
}
]
studentDB> db.students.find(
|   course: "MERN Stack"
| })
[
  {
    _id: ObjectId('6ac13020b76bbb7d310d6154'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ac13020b76bbb7d310d6156'),
    name: 'Shisue',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
```

## 6. Update One Document

```
studentDB> db.students.updateOne(  
|   { name: "Xavier" },  
|   { $set: { status: "completed" } }  
| )  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

## 7. Update Multiple Documents

```
studentDB> db.students.updateMany(  
|   { course: "MERN Stack" },  
|   { $set: { status: "completed" } }  
| )  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 2,  
  modifiedCount: 2,  
  upsertedCount: 0  
}
```

## 8. Delete One Document

## 9. Delete Multiple Documents

```
studentDB> db.students.deleteOne({
  name: "Jasmine"
})
{ acknowledged: true, deletedCount: 1 }
studentDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 4 }
studentDB> db.students.insertMany([
  {
    name: "Arun",
    age: 21,
    course: "MERN Stack",
    status: "ongoing"
  },
  {
    name: "Kavin",
    age: 22,
    course: "Java",
    status: "completed"
  },
  {
    name: "Rahul",
    age: 19,
    course: "MERN Stack",
    status: "ongoing"
  },
  {
    name: "Vijay",
    age: 23,
    course: "Python",
    status: "completed"
  }
])
```

## 10. MongoDB Query Operators

\$gt

```
studentDB> db.students.find({
|   age: { $gt: 20 }
| })
[
  {
    _id: ObjectId('6ac139c5b76bbb7d310d6158'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ac139c5b76bbb7d310d6159'),
    name: 'Kavin',
    age: 22,
    course: 'Java',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac139c5b76bbb7d310d615b'),
    name: 'Vijay',
    age: 23,
    course: 'Python',
    status: 'completed'
  }
]
```

## \$lt and \$in

```
]
studentDB> db.students.find({
|   age: { $lt: 21 }
| })
[
|   {
|     _id: ObjectId('6ac139c5b76bbb7d310d615a'),
|     name: 'Rahul',
|     age: 19,
|     course: 'MERN Stack',
|     status: 'ongoing'
|   }
| ]
studentDB> db.students.find({
|   course: { $in: ["MERN Stack", "Java"] }
| })
[
|   {
|     _id: ObjectId('6ac139c5b76bbb7d310d6158'),
|     name: 'Arun',
|     age: 21,
|     course: 'MERN Stack',
|     status: 'ongoing'
|   },
|   {
|     _id: ObjectId('6ac139c5b76bbb7d310d6159'),
|     name: 'Kavin',
|     age: 22,
|     course: 'Java',
|     status: 'completed'
|   },
|   {
|     _id: ObjectId('6ac139c5b76bbb7d310d615a'),
|     name: 'Rahul',
|     age: 19,
```

## \$and and \$or

```
studentDB> db.students.find({
|   $and: [
|       { age: { $gt: 20 } },
|       { status: "completed" }
|   ]
| })
[
  {
    _id: ObjectId('6ac139c5b76bbb7d310d6159'),
    name: 'Kavin',
    age: 22,
    course: 'Java',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac139c5b76bbb7d310d615b'),
    name: 'Vijay',
    age: 23,
    course: 'Python',
    status: 'completed'
  }
]
studentDB> db.students.find({
|   $or: [
|       { course: "MERN Stack" },
|       { course: "Java" }
|   ]
| })
[
  {
    _id: ObjectId('6ac139c5b76bbb7d310d6158'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
```

\$exists

```
studentDB> db.students.find({
|   status: { $exists: true }
| })
[
  {
    _id: ObjectId('6ac139c5b76bbb7d310d6158'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ac139c5b76bbb7d310d6159'),
    name: 'Kavin',
    age: 22,
    course: 'Java',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac139c5b76bbb7d310d615a'),
    name: 'Rahul',
    age: 19,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ac139c5b76bbb7d310d615b'),
    name: 'Vijay',
    age: 23,
    course: 'Python',
    status: 'completed'
  }
]
```

## 11. Library System Use Case

```
studentDB> use libraryDB
switched to db libraryDB
libraryDB> db.createCollection("books")
{ ok: 1 }
libraryDB> db.books.insertMany([
|   {
|     title: "The Alchemist",
|     author: "Paulo Coelho",
|     category: "Fiction",
|     available: true
|   },
|   {
|     title: "Atomic Habits",
|     author: "James Clear",
|     category: "Self Help",
|     available: true
|   },
|   {
|     title: "Clean Code",
|     author: "Robert Martin",
|     category: "Programming",
|     available: false
|   }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac13ab4b76bbb7d310d615c'),
    '1': ObjectId('6ac13ab4b76bbb7d310d615d'),
    '2': ObjectId('6ac13ab4b76bbb7d310d615e')
  }
}
```

## 12. Search Books

## 13. Update Book Availability

```
libraryDB> db.books.find({
|   category: "Programming"
| })
[
|   {
|     _id: ObjectId('6ac13ab4b76bbb7d310d615e'),
|     title: 'Clean Code',
|     author: 'Robert Martin',
|     category: 'Programming',
|     available: false
|   }
| ]
libraryDB> db.books.updateOne(
|   { title: "The Alchemist" },
|   { $set: { available: false } }
| )
{
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
```